

[L0-L1 核心本质与设计逻辑]

L0 一句话本质：计算机网络自顶向下系统是一个以套接字为用户态边界，依靠传输层滑动窗口与动态拥塞控制提供高可靠/高吞吐传输，通过网络层最长前缀匹配查表与自治系统 BGP 协议路由转发，并在链路层利用 CSMA 多路访问解决共享介质碰撞冲突的复杂分布式通信体系。

L1 四句话逻辑：

- 应用抽象与套接字边界：**应用层通过统一的 Socket 套接字描述符（基于文件抽象）向下递交数据流；DNS 协议通过分布式级联查询将主机名转换为 IP 地址，为传输建立物理导航。
- 双端滑动窗口与超时重传：**传输层采用滑动窗口机制平衡发送速率与接收缓冲区限额；通过不断采样的物理 RTT 动态计算安全超时期（RTO），当报文丢失时触发 GBN 或 SR 重传，保证物理信道不可靠时的逻辑可靠。
- 拥塞控制从 Cubic 到 BBR 的演进：**传统 Cubic 以丢包作为拥塞触发信号，通过三次方曲线迅速填满网络链路缓冲区，导致排队延迟剧增；现代 BBR 算法放弃丢包驱动，改用主动测量瓶颈带宽与最小 RTT 的交点（Kleinrock 限界），实现最高吞吐与零队列积压。
- 分级自治路由与介质争用：**网络层在自治系统内部使用 OSPF/Dijkstra 路由，在自治系统之间使用 BGP 路径属性控制流量走向；链路层在共享总线上采用 CSMA/CD 监听碰撞，而在无线介质中则通过 RTS/CTS 与 CSMA/CA 预约信道，消除多路冲突。

[L2 核心数据流流转拓折]

• 用户调用 (write) □ 文件描述符层 (fwrite) □ 管道层 (pipewrite/唤醒读进程) 或 Inode 层 (writei) □ 日志层 (write_log) □ 块缓存层 (bwrite) □ 磁盘驱动层 (virtio_disk_rw) □ 硬件物理磁盘。

[M1.1] HTTP/1.x 串行阻塞与 HTTP/2 双向帧复用机制 (‘HTTP/1.1’ vs ‘HTTP/2’)

- 队头阻塞 (HOL)：**HTTP/1.1 支持管道化 (Pipelining)，但响应必须严格按请求顺序返回，慢响应会导致后续请求积压。
- 二进制分帧：**HTTP/2 将报文拆分为 HEADERS 帧和 DATA 帧，并在帧头嵌入流标识符 (Stream ID)。
- 双向多路复用：**多个流并发交错复用单条 TCP 连接，接收端根据 Stream ID 重组，消除了应用层队头阻塞。

[M1.2] DNS 递归与迭代树形层次查询流控逻辑 (‘DNS Hierarchy’)

- 双重查询控制：**
- 递归查询：**客户端向本地 DNS 服务器发起请求，本地域名服务器必须代为查出最终结果并返回。
- 迭代查询：**本地域名服务器依次查询根域名服务器 (Root)、顶级域名服务器 (TLD) 和权威域名服务器 (Authoritative)，各级服务器返回下一步应查询的服务器 IP。
- 端口与传输限制：**默认使用 UDP 53 端口；若响应报文长度超过 512 字节，DNS 报文首部 TC (Truncated) 位置 1，客户端自动切换为 TCP 53 端口重新发起查询。

[M1.3] BSD Socket API 执行序列与内核底层连接映射 (‘net/socket.c’)

- 服务端建立三部曲：**socket() 建立文件描述符 □ bind() 绑定物理 IP 与端口 □ listen() 开启监听并配置 backlog 半连接与全连接队列表长度。
- 连接接管与阻塞：**accept() 阻塞等待，直到 TCP 三次握手成功，将连接从全连接队列入队并返回一个全新的套接字描述符。
- 客户端激活：**connect() 主动发起 SYN 握手，触发内核协议栈状态转为 SYN_SENT。

[M1.4] epoll Multiplexing epoll 边缘/水平触发 (‘fs/eventpoll.c’)

- O(1) 事件检索：**epoll 放弃了 select/poll 对描述符集合的 O(N) 线性遍历，在内核维护红黑树追踪监测 Socket，并使用双向链表 (Ready List) 存放就绪事件。
- 触发模式对比：**
- 水平触发 (LT)：**只要缓冲区有未读数据，epoll_wait 就会持续触发通知。
- 边缘触发 (ET)：**仅在缓冲区状态改变（如数据到达）时触发一次。要求套接字必须为非阻塞 (Non-blocking)，且必须循环读取直至返回 EAGAIN，以防漏读。

[M2.1] 传输层解复用机制与双端端口绑定规则 (‘net/ipv4/af_inet.c’)

- 无连接解复用 (UDP)：**根据二元组（目的 IP、目的端口）分发报文，不同源端发往相同目的端口的报文进入同一个 Socket 队列。
- 面向连接解复用 (TCP)：**根据四元组（源 IP、源端口、目的 IP、目的端口）分发。
- 多 Socket 监听：**多路并发连接可以共享相同的目的端口，内核通过哈希表（Established HashTable）精准定位到各个不同的 Socket 实体。

[M2.2] 滑动窗口回退 N 步 (GBN) 状态机与累积确认限制 (‘net/ipv4/tcp_input.c’)

- 发送窗口控制：**允许发送方发送最多 N 个未确认报文，由一个全局定时器监控最早发送的未确认块。
- 累积确认 (Cumulative ACK)：**接收方只发送当前有序到达的最大序号确认 (ACK y 代表 y 及以前的报文均已收受)，丢弃所有失序到达的报文。
- 超时回退：**一旦发生超时，发送方必须重传窗口内所有已发送但未确认的 N 个报文，吞吐量受高延迟链路影响剧烈。

[M2.3] 选择重传 (SR) 乱序缓存与接收窗口帧步对齐 (‘net/ipv4/tcp_input.c’)

- 单包定时器：**发送方为每个发送的报文维护独立的定时器，超时仅重传丢失的单包，不影响后续已发送报文。
- 接收端乱序缓存：**接收方窗口大小同样为 N （满足 $N \leq 2^{20} - 1$ 以免混淆新旧序号），接收并缓存失序报文，回送非累积的独立确认 (ACK)。
- 窗口滑动对齐：**当且仅当接收窗口的基序号（当前最小未收报文）被填补并滑动时，接收端才向上层递交数据。

[M2.4] RTT 动态估计与超时期 (RTO) 指数加权移动平均算法 (EWMA) (‘net/ipv4/tcp_input.c’)

- 平滑 RTT 采样：**通过发送与 ACK 的时间差测量 SampleRTT。
- EstimatedRTT = $(1 - \alpha) \times \text{EstimatedRTT} + \alpha \times \text{SampleRTT}$ ($\alpha = 0.125$)**
- 偏差与波动率计算：**
- DevRTT = $(1 - \beta) \times \text{DevRTT} + \beta \times |\text{SampleRTT} - \text{EstimatedRTT}|$ ($\beta = 0.25$)**
- 安全 RTO 计算：** $\text{TimeoutInterval} (RTO) = \text{EstimatedRTT} + 4 \times \text{DevRTT}$ 。当重传超时触发时，RTO 强制翻倍（指数退避），以降低网络负荷。

[M2.5] TCP 三次握手建立与四次挥手半关闭状态转移矩阵 (‘net/ipv4/tcp_input.c’)

- 建立状态机：**
- Client CLOSED □ 发送 SYN □ SYN_SENT □ 收到 SYN+ACK 并发回 ACK □ ESTABLISHED。**
- Server LISTEN □ 收到 SYN 并回送 SYN+ACK □ SYN_RECV □ 收到 ACK □ ESTABLISHED。**
- 断开半关闭状态机：**
- 主动关闭方：ESTABLISHED □ 发送 FIN □ FIN_WAIT_1 □ 收到 ACK □ FIN_WAIT_2 □ 收到 FIN 并发回 ACK □ TIME_WAIT (等待 2MSL 以确保残留报文消亡) □ CLOSED。**
- 被动关闭方：ESTABLISHED □ 收到 FIN 并发回 ACK □ CLOSE_WAIT □ 准备完毕发送 FIN □ LAST_ACK □ 收到最后 ACK □ CLOSED。**

[M3.1] 经典 AIMD 锯齿波与慢启动/拥塞避免转换阀门 (‘net/ipv4/tcp_cong.c’)

- 慢启动 (Slow Start)：**拥塞窗口 $cwnd$ 从 1 MSS 开始，每收到一个 ACK，窗口大小加 1，即每个 RTT 内 $cwnd$ 呈指数翻倍，直到 $cwnd \geq ssthresh$ 。
- 拥塞避免 (Congestion Avoidance)：** $cwnd$ 限制为每个 RTT 仅增加 1 MSS（加性增，Additive Increase）。
- 阈值阀门判定：**一旦发生丢包（超时触发），将慢启动阈值 $ssthresh$ 置为当前 $cwnd/2$ ，并将 $cwnd$ 重置为 1 MSS，重新进入慢启动。

[M3.2] 快速重传与快速恢复机制 (FRR / RFC 5681) (‘net/ipv4/tcp_input.c’)

- 三重重复确认触发：**当接收端检测到报文段失序，会立即连续回送冗余的确认 (Duplicate ACK)。发送端一旦累积收到 3 个冗余确认（即 4 个相同 ACK），判定该包丢失，立即重传该包，无需等待定时器超时（快速重传）。
- 快速恢复 (Fast Recovery)：**避免将 $cwnd$ 跌落至 1。将 $ssthresh$ 设为 $cwnd/2$ ，配置 $cwnd = ssthresh + 3 \times MSS$ ，收到新数据确认后将 $cwnd$ 降回到 $ssthresh$ ，直接进入拥塞避免状态。

[M3.3] Cubic 拥塞算法三次方控制曲线与公平性限界 (‘net/ipv4/tcp_cubic.c’)

- 三次方增长函数：**Cubic 放弃了传统线性窗口增长，窗口大小 $W(t)$ 与上次拥塞减少以来的时间 t 呈三次方关系：

$$W(t) = C(t - K)^3 + W_{\max}$$

其中 $K = \sqrt[3]{W_{\max} \times (1 - \beta) / C}$ 。

- 控制曲线特征：**在接近上次发生拥塞的窗口大小 $W_{\max}$ 时，窗口增长极度扁平平缓，提高稳定性；远离该区域后迅速加速增长以探测链路新带宽，完美兼顾高带宽时延迟链路的公平性与吞吐。

[M3.4] Google BBR 算法瓶颈带宽/最小 RTT 估算与 Kleinrock 限界控制 (‘net/ipv4/tcp_bbr.c’)

- 非丢包驱动革命：**Cubic 常因填满物理缓冲区触发丢包而导致“缓冲区膨胀” (Bufferbloat)。BBR 通过主动测量两个核心指标来控制发送速率：
- 瓶颈带宽 (BtlBw)：**滑动窗口内最大数据投速率。
- 最小传播时延 (RTprop)：**过滤掉排队延迟后的物理 RTT。
- Kleinrock 限界控制：**维持在 Kleinrock 最优工作点：发送速率 $PacingRate = BtlBw$ ，管中数据量 $Inflight = BtlBw \times RTprop$ （即带宽时延积 BDP）。这使得网络吞吐最大化，且物理队列深度为 0，消除了延迟波动。

[壹] 计算机网络协议栈、多路同步与物理传输折衷矩阵

- ✎ **无限大发送窗口：**认为发送窗口 $cwnd$ 越大，数据投速率就越高。✎ **物理路由器缓存溢出：**过大窗口会导致报文在路由队列入队积压导致丢包，诱发拥塞崩溃。→ **双限制窗口约束：**发送限制 $W < \min(cwnd, rwnd)$ ，同时受制于拥塞限额与接收能力。
- ✎ **高限时重传：**认为定时器超时期 (RTO) 设置得越小，丢失报文就能越快重传。✎ **网络往返延迟抖动：**过小的 RTO 会导致大量正常传输但延迟的包被误判重传，造成带宽浪费。→ **动态 EWMA 算法 + 4 倍方差裕量：**基于 RTT 均值与方差动态计算 RTO，平衡重传时效与误判率。
- ✎ **丢包等同拥塞：**认为只要网络发生丢包，就必须立刻把拥塞窗口减半以防拥塞。✎ **无线物理信道高随机误码率：**无线电磁干扰丢包与网络拥塞排队溢出物理机制完全不同。→ **BBR 速率步调控制 (Pacing)：**以物理瓶颈带宽为基准调整发送速率，对随机非拥塞丢包具有强鲁棒性。
- ✎ **零队列延迟抵退：**为了追求极致低延迟，要求所有路由交换设备的缓存区大小全部缩减为零。✎ **突发流量排队冲突：**缺乏缓冲区会导致任何突发流量（Burst）因无法排队而直接丢弃。→ **Rule of Thumb 限界缓冲：**路由路由器端口缓存大小设为 $C \times RTT / \sqrt{N}$ ，平衡吞吐量与排队时延。

[M4.1] IPv4 首部分片机制与 IPv6 简化流标记首部对比 ('net/ipv4/ip_input.c')

- **IPv4 分片控制**: 通过 16 位标识符 (Identification)、3 位标志位 (Flags: DF 不分片, MF 更多分片) 以及 13 位片偏移 (Fragment Offset) 在传输路径上对超过 MTU 的报文进行物理分片, 并在目的主机重组。
- **IPv6 报头简化**: 取消了路由器分片功能 (要求源端必须通过路径 MTU 发现预先分片), 首部大小固定为 40 字节。移除了首部校验和以提高转发速率, 并新增 20 位流标记 (Flow Label), 用于加速特定多媒体流的分类路由处理。

[M4.2] 硬件级最长前缀匹配 (LPM) 二叉 Trie 树查找算法 ('net/ipv4/fib_trie.c')

- **最长前缀匹配 (LPM)**: 路由转发表使用 CIDR 斜线记法, 一个目的 IP 可能匹配多个子网掩码前缀, 必须选择最长匹配的前缀转发。
- **Trie 树优化结构**: 采用 Level-Compressed Trie (LC-Trie) 树, 通过路径压缩和子树压缩, 将二叉树变为多路查找树, 减少内存访问次数。
- **硬件实现**: 核心路由器使用三态内容寻址寄存器 TCAM (Ternary Content Addressable Memory), 在一拍时钟内并执行匹配所有的 0、1 和 Wildcard (*) 标志, 实现 O(1) 的超高速转发速度。

[M4.3] 路由器内部横跨条 (Crossbar) 交换矩阵与头部阻塞 (HOL) ('Router Switch')

- **横跨条交换**: 非阻塞交换网络, 由 $N \times N$ 条半导体交叉线组成, 允许 N 个输入端口与 N 个输出端口并发无冲突传输。
- **队头阻塞 (HOL)**: 如果输入队列开头的两个报文同时竞争同一个输出端口, 其中一个会被阻塞在输入队列开头。这会导导致该队列中后续发往其他空闲输出端口的报文也无法向前移动, 限制了交换性能。
- **解决方案**: 输入端使用虚拟输出队列 VOQ (Virtual Output Queuing), 将一个物理队列拆分为面向不同输出端口的 N 个逻辑队列。

[M4.4] ICMP 差错报文控制协议与 Traceroute 物理寻径原理 ('net/ipv4/icmp.c')

- **ICMP 差错控制**: IP 协议之上 (协议号 1) 的控制协议, 用于报告网络异常。类型 3 代表目的不可达, 类型 11 代表超时 (TTL Exceeded)。
- **Traceroute 物理寻径**:
 1. 源端发送一个目的地为高端口号的不可达 UDP 报文, 初始 TTL 设为 1。
 2. 第一跳路由器将 TTL 减 1 归零, 丢弃该报文, 向源端回送一个 ICMP 差错报文 (Type 11 Code 0)。
 3. 源端获取第一跳路由器的 IP 地址。
 4. 递增 TTL (2, 3...) 重复发送, 直到报文到达目的主机。目的主机由于端口不可达, 回送 ICMP Port Unreachable 报文 (Type 3 Code 3), 宣告路由追踪结束。

[M5.1] 链路状态路由算法 (Dijkstra) 与 OSPF 洪泛同步 ('OSPF Daemon')

- **全局信息路由**: 要求所有路由器掌握完整的网络拓扑。
- **OSPF 洪泛协议**: 路由器向自治系统内所有其他路由器洪泛链路状态通告 (LSA), 通告自身与邻居的链路代价。
- **LSDB 同步与计算**: 各路由通过可靠确认同步链路状态数据库 (LSDB)。同步完成后, 本地独立运行 Dijkstra 最短路径算法, 构建以自身为根节点的单源最短路径树, 输出一跳路由。

[M5.2] 距离向量路由算法 (Bellman-Ford) 环路与无穷大收敛问题 ('RIP Daemon')

- **局部迭代路由**: 基于 Bellman-Ford 迭代方程: $D_x(y) = \min_v \{c(x, v) + D_v(y)\}$, 路由器仅与邻居交换路由表向量。
- **无穷计数灾难**: 当某条链路时变差或中断, 邻居间的坏消息会以迭代更新的形式在路由表间来回传递, 导致“路由环路”并缓慢收敛至无穷大 (Count-to-Infinity)。
- **防环策略**: 使用水平分割 (Split Horizon, 不向邻居通告从该邻居学来的路由) 以及毒性逆转 (Poison Reverse, 将该路由代价值设为 16 宣告不可达)。

[M5.3] 边界网关协议 (BGP) 路径属性 (AS-Path, Next-Hop, Local-Pref) ('BGP Daemon')

- **AS 自治系统路由**: BGP 负责自治系统之间的路径矢量路由选择, 避免环路并支持商业策略控制。
- **核心路径属性**:
 - **AS-Path**: 记录路由由通告所经过的 AS 号列表。当收到路由由通告中包含自身 AS 号, 判定为环路并直接丢弃。
 - **Next-Hop**: 到达下一个 AS 的入口边界路由器 IP 地址。
 - **Local-Preference**: 自治系统内部的政策首选项, 用于控制出站流量的网关选择, 值越高越优先。

[M5.4] 自治系统 (AS) 之间多准则 BGP 决策决策树过滤规则 ('BGP Engine')

- **BGP 过滤决策树**: 当路由器收到多条去往同一目的地的 BGP 路由, 按照以下决策规则进行唯一筛选:
 1. 选择具有最高本地优先级 (Local-Pref) 的路由。
 2. 选择具有最短自治系统路径 (AS-Path) 长度的路由。
 3. 比较源起源 (Origin): IGP 优于 EGP, EGP 优于 Incomplete。
 4. 选择具有最低多出鉴别器 (MED, 控制入站流量) 值的路由。
 5. 优先选择从外部 BGP (eBGP) 获知的路由, 而非内部 BGP (iBGP)。
 6. 选择到 Next-Hop 边界路由器具有最近 IGP 内部路径代价的路由。
 7. 最终保留具有最小 BGP 路由器 ID 的路由。

[M5.5] 内部 BGP (iBGP) 与外部 BGP (eBGP) 级联注入流向 ('BGP Peering')

- **外部对等体 (eBGP)**: 两个处于不同 AS 的边界路由器之间的 BGP 连接, 用于跨自治系统通告外部可达路径。
- **内部对等体 (iBGP)**: 同一个 AS 内部路由器之间的 BGP 连接, 用于在自治系统内部同步和分发从 eBGP 获知的外部路由。
- **水平分割约束**: 为防止 AS 内部的 BGP 路由环路, iBGP 规定: 通过 iBGP 学到的路由不能再通告给其他 iBGP 邻居。这要求 AS 内部所有运行 BGP 的路由器必须建立 **全网状连接 (Full Mesh)**, 或使用路由反射器 (Route Reflector) 进行集中同步。

[M5.6] 软件定义网络 (SDN) 控制面与数据面 OpenFlow 流表解耦 ('SDN Architecture')

- **数控分离解耦**: 将网络设备的路由计算逻辑 (控制面) 从交换芯片转发硬件 (数据面) 中剥离, 交由集中的 SDN 控制器管理。
- **OpenFlow 三元组流表**: SDN 交换机根据多级流表 (Flow Table) 转发, 每个流表项由三部分组成:
 1. **匹配域 (Match Fields)**: 支持跨层匹配 (入端口、源/目的 MAC、源/目的 IP、TCP/UDP 端口)。
 2. **动作 (Actions)**: 匹配成功后的执行操作 (Forward 转发、Drop 丢弃、Modify 字段修改、GoTo 触发下一级流表)。
 3. **计数器 (Counters)**: 统计匹配报文数量, 用于性能监控与计费。

[M6.1] 链路层成帧、奇偶校验与 CRC 循环冗余多项式校验 ('net/core/')

- **帧封装 (Framing)**: 在网络层 IP 数据报两端加入首部 (Header) 和尾部 (Trailer), 构建链路层帧。
- **CRC 循环冗余校验**: 发送端在 d 位数据 D 后附加 r 位校验码 R 。
 - 选取生成多项式 G (共 $r + 1$ 位)。
 - 使附加后的数据 $(D \cdot 2^r) \oplus R$ 能够被 G 在模 2 算术下整除。
 - 接收端对收到的帧进行模 2 除以 G 计算, 若余数不为零, 判定帧损坏并直接丢弃该帧。

[M6.2] 共享总线以太网 CSMA/CD 载波监听与指数退避冲突避免 ('drivers/net/')

- **载波监听与碰撞检测**: 发送前监听信道 (Carrier Sense), 若忙则等待, 若闲则发送; 发送的同时持续监听信道 (Collision Detection)。
- **碰撞中止**: 若检测到碰撞, 立即停止发送数据, 并发送一个 48 位强化干扰 jam 信号, 以确保总线上所有其他节点都知道发生了碰撞。
- **二进制指数退避 (BEB)**: 发生第 n 次碰撞后, 随机等待 $K \times 512$ 位时间, 其中 K 随机取自区间 $[0, 2^m - 1]$ ($m = \min(n, 10)$)。当碰撞次数超过 16 次, 向上层报告传输错误。

[M6.3] 无线局域网 CSMA/CA 碰撞避免、RTS/CTS 信道预约控制 ('net/mac80211/')

- **无线碰撞无法检测**: 由于无线网卡发送时的电磁波强度远大于接收到的信号, 无线节点无法边发边听进行碰撞检测。
- **碰撞避免机制 (CSMA/CA)**: 发送端监听信道若忙, 在 IFS (帧间间隔, 通常为 DIFS) 后执行随机退避; 若闲, 则发送完整帧, 接收端必须显式回复 ACK。
- **信道预约机制 (RTS/CTS)**: 为解决“隐蔽终端”冲突:
 1. 源端发送请求发送帧 RTS (Request to Send), 指明数据传输所需时间。
 2. 目的端回复允许发送帧 CTS (Clear to Send)。
 3. 周边其他所有无线节点在听到 RTS 或 CTS 后, 在规定时间内 (NAV, 网络分配向量) 保持沉默, 实现物理信道预约。

[M6.4] ARP 地址解析协议广播查询与缓存表动态老化 ('net/ipv4/arp.c')

- **MAC/IP 动态翻译**: 在局域网内传输帧必须知道目的主机的物理 MAC 地址。ARP 用于将 32 位 IP 地址解析为 48 位 MAC 地址。
- **广播查询与单播回复**:
 - 若本地 ARP 缓存表未命中, 源端发送 ARP 请求帧, 目的 MAC 设为广播地址 FF:FF:FF:FF:FF:FF。
 - 局域网内所有主机接收该帧, 仅 IP 匹配的主机回送单播 ARP 响应帧, 告知其 MAC 地址。
- **缓存老化**: 源端缓存该映射, 并配置生存期 TTL (通常为 10 到 20 分钟), 超时后自动清除以适应网卡更换。

[M6.5] VLAN 虚拟局域网 802.1Q 标签注入与交换机自学习算法 ('net/8021q/')

- **广播域逻辑隔离**: 通过在单台物理交换机上配置 VLAN 标识 (VLAN ID), 将一个物理局域网划分为多个逻辑上的独立广播域。
- **802.1Q 帧格式**: 在以太网源 MAC 和类型字段之间插入 4 字节的 802.1Q 标签 (Tag), 包含 12 位 VLAN ID, 用于在交换机之间的级联干线 (Trunk) 链路上传输。
- **交换机自学习**: 交换机监听端口进来的帧, 将 (源 MAC 地址、端口号、VLAN ID) 存入 MAC 地址表并配置老化时间。若目的 MAC 未在表中, 则在该 VLAN 的所有其他端口广播转发; 若匹配, 则单播定向转发。

[贰] top-down-networking 底层诊断与网络性能调优 CLI 工具字典	
T1: 计算机网络内核网络性能调优核心参数对照表 net.ipv4.tcp_congestion_control: 指定 TCP 使用的拥塞控制算法。常用值包括 cubic (默认) 与 bbr (需内核版本 4.9+ 且配置 fq 队列排队规则)。 net.core.rmem_max/net.core.wmem_max: 设置系统全局 socket 读/写缓存区的最大物理限制 (字节)。高吞吐长肥管道 (High BDP) 必须调大此参数。 net.ipv4.tcp_rmem/net.ipv4.tcp_wmem: TCP 自适应读/写缓存的三个值配置: min (最小保障)、default (默认分配)、max (最大上限, 受全局 wmem 限制)。 net.ipv4.tcp_max_syn_backlog: TCP 半连接队列 (SYN_RECV 状态) 的最大长度。防止 SYN Flood 洪水攻击时需调大此参数或开启 net.ipv4.tcp_syncookies。 net.core.somaxconn: TCP 全连接队列 (等待用户 accept) 的最大长度。高并发高吞吐服务器必须将此值与 Nginx 的 backlog 同时调大 (通常调至 2048+)。	T2: 计算机网络全栈排查与性能剖析 CLI 指令字典 lsof: 1. 抓取物理网卡网络原始报文并保存为 pcap 文件 (使用 tcpdump) 2. 交互式实时监听网络流量与进程套接字详情 (使用 tshark/wireshark-cli) 3. 查看系统当前 TCP 连接详细状态、滑动窗口与 RTT 估算 (使用 ss -i) 4. 路由与多联点物理链路诊断 (使用 traceroute / mtr) 5. 级联分析每一跳路由的丢包率、延迟抖动与 ICMP 响应时间 6. 分析网卡多路访问统计与丢包计数 (使用 ip -s / ethtool) 7. 显示以太网帧错误计数、CRC 校验失败及碰撞冲突 (collisions) 次数 8. 查看物理网卡驱动层及 MAC 控制器的底层统计计数